

# Student Task Manager System

## Project Report

Course: CSE-3532 — Tools and Technologies for Internet Programming

Semester: Spring 2026 | 5th Semester | Section 5CM

Institution: International Islamic University Chittagong (IIUC)

### Submitted by:

Mehedi Hasan Howlader (C241086)

Mohammad Sadman Tahiat (C241100)

Minhaj Hasan Rohan (C241101)

### Submitted To:

**Ahasanul Kalam Akib**

Adjunct Lecturer

Department of CSE, IIUC

## Abstract

This project presents a Student Task Manager System, a full-stack web application built with the MERN Stack (MongoDB, Express.js, React.js, Node.js) to help university students manage their academic tasks efficiently. The system provides a centralized dashboard where students can create, edit, delete, and track tasks with deadlines, priority levels, and status indicators. It features secure authentication via email/password and Google OAuth (Firebase), interactive analytics with Chart.js visualizations, a responsive dark/light theme UI, and smart filtering, search, and sorting capabilities. The application addresses the real problem that 67% of university students struggle to organize academic tasks, leading to missed deadlines and lower academic performance. The system was developed as a semester project for the course CSE-3532 at International Islamic University Chittagong (IIUC).

## 1. Introduction

University students face increasingly demanding academic workloads. Managing multiple courses, assignments, lab reports, presentations, and personal responsibilities simultaneously is a significant challenge. Traditional methods such as scattered paper notes, generic reminder apps, or relying on memory do not provide structured priority assignment, deadline tracking, or centralized progress visualization.

The Student Task Manager System is a purpose-built web application designed to solve these challenges. It provides a centralized, intuitive, and modern dashboard where students can manage all their academic tasks from a single interface accessible on any device. The application uses the MERN stack along with Tailwind CSS for styling, Chart.js for analytics, and Firebase for Google authentication.

## 2. Problem Definition

### 2.1 Problem Statement

University students struggle to organize their academic tasks effectively. Existing solutions are either too generic (like to-do apps not designed for academic contexts) or too fragmented (using different tools for different purposes). There is a lack of a centralized, student-specific task management tool that combines task creation, deadline tracking, priority management, progress visualization, and analytics in one place.

### 2.2 Why This Project?

**67% of university students struggle with task organization, leading to missed deadlines.**

Traditional methods (paper notes, memory, generic apps) lack structured priority and deadline tracking.

No single tool is designed specifically for the academic workflow of university students.

Students need a responsive, accessible tool that works on any device (mobile, tablet, desktop).

### 3. Objectives

- Develop a full-stack MERN web application for student task management.
- Implement secure user authentication with email/password and Google OAuth via Firebase.
- Provide full CRUD (Create, Read, Update, Delete) operations on academic tasks.
- Enable task attributes: title, description, deadline, priority (Low/Medium/High), and status (Pending/In Progress/Completed).
- Build a productivity dashboard with summary statistics and interactive charts (Chart.js).
- Implement smart search, filtering, and sorting for efficient task retrieval.
- Design a responsive, accessible UI with dark/light theme support using Tailwind CSS.
- Ensure the application works seamlessly across mobile, tablet, and desktop devices.

## 4. System Analysis

### 4.1 Functional Requirements

| ID    | Requirement         | Description                                                      |
|-------|---------------------|------------------------------------------------------------------|
| FR-01 | User Registration   | Users can create an account with name, email, and password       |
| FR-02 | User Login          | Email/password login and Google OAuth (Firebase)                 |
| FR-03 | Task Creation       | Create tasks with title, description, deadline, priority, status |
| FR-04 | Task Listing        | View all tasks in grid/list layout for authenticated users       |
| FR-05 | Task Editing        | Edit any field of an existing task                               |
| FR-06 | Task Deletion       | Permanently delete a task with confirmation dialog               |
| FR-07 | Status Management   | Cycle status: Pending → In Progress → Completed                  |
| FR-08 | Search Tasks        | Real-time text search by task title                              |
| FR-09 | Filter Tasks        | Filter by status and priority level                              |
| FR-10 | Sort Tasks          | Sort by deadline (asc/desc) or creation date                     |
| FR-11 | Analytics Dashboard | Summary cards + Doughnut/Bar charts (Chart.js)                   |

|              |                  |                                                 |
|--------------|------------------|-------------------------------------------------|
| <b>FR-12</b> | Theme Switcher   | Dark/Light toggle with localStorage persistence |
| <b>FR-13</b> | Protected Routes | Unauthenticated users redirected to Login page  |
| <b>FR-14</b> | User Profile     | Display logged-in user name on dashboard        |
| <b>FR-15</b> | Logout           | Clear stored tokens and user data on logout     |

## 4.2 Non-Functional Requirements

| ID            | Requirement     | Description                                              |
|---------------|-----------------|----------------------------------------------------------|
| <b>NFR-01</b> | Performance     | API responses within 500ms under normal load             |
| <b>NFR-02</b> | Security        | bcryptjs hashing (12 rounds), JWT stateless auth         |
| <b>NFR-03</b> | Responsiveness  | Mobile (320px+), tablet (768px+), desktop (1024px+)      |
| <b>NFR-04</b> | Usability       | Intuitive UI with minimal onboarding, consistent design  |
| <b>NFR-05</b> | Compatibility   | Modern browsers (Chrome, Firefox, Safari, Edge)          |
| <b>NFR-06</b> | Scalability     | MongoDB Atlas + stateless backend for horizontal scaling |
| <b>NFR-07</b> | Maintainability | Modular code with separation of concerns                 |
| <b>NFR-08</b> | Availability    | 99% availability target during development lifecycle     |
| <b>NFR-09</b> | Accessibility   | Semantic HTML, ARIA labels, keyboard navigation          |

## 5. System Design

### 5.1 ER Diagram (Entity Relationship Diagram)

#### User (1) — (N) Task

User Entity: `_id` (PK), name, email (unique, lowercase), password (bcrypt-hashed), createdAt, updatedAt

Task Entity: `_id` (PK), user (FK → User.`_id`), title, description, deadline, priority (Low/Medium/High), status (Pending/In Progress/Completed), createdAt, updatedAt

Relationship: One User has Many Tasks (1:N). Each Task belongs to exactly one User. All task queries are filtered by user.\_id using JWT authentication.

## 5.2 Data Flow Diagram (DFD)

### Level 0 (Context Diagram)

**Actor: Student (Browser)**

Student → Student Task Manager System (Frontend React + Backend Express + MongoDB + Firebase Auth)

External Entities: MongoDB Atlas (database), Firebase Authentication (Google OAuth)

### Level 1 (Detailed Processes)

Process 1.0 — Authentication: Student → Register/Login → Backend → MongoDB (User) / Firebase Auth → JWT returned → localStorage

Process 2.0 — Task Management: Student → CRUD Operations → Backend → MongoDB (Task, filtered by user.\_id) → JSON response → UI Render

Process 3.0 — Analytics: Student → Stats Request → Backend → Aggregates Task data by status/priority → Stats JSON → Chart.js Charts

Process 4.0 — Theme & UI: Student toggles theme → Context API → CSS variables change → localStorage persistence

## 6. Technology Used

### Frontend

| Technology       | Version  | Purpose                     |
|------------------|----------|-----------------------------|
| React            | ^18.3.1  | UI component library        |
| Vite             | ^8.0.16  | Build tool and dev server   |
| Tailwind CSS     | ^3.4.13  | Utility-first CSS framework |
| React Router DOM | ^6.26.2  | Client-side routing         |
| Axios            | ^1.7.7   | HTTP client for API calls   |
| Chart.js         | ^4.4.4   | Data visualization library  |
| React ChartJS 2  | ^5.2.0   | React wrapper for Chart.js  |
| Firebase         | ^12.15.0 | Google Authentication       |

## Backend

| Technology   | Version     | Purpose                           |
|--------------|-------------|-----------------------------------|
| Node.js      | Runtime     | JavaScript server runtime         |
| Express.js   | ^4.21.0     | Web framework for REST API        |
| MongoDB      | Cloud/Local | NoSQL document database           |
| Mongoose     | ^8.6.0      | ODM for MongoDB                   |
| bcryptjs     | ^2.4.3      | Password hashing                  |
| jsonwebtoken | ^9.0.2      | JWT token generation/verification |
| CORS         | ^2.8.5      | Cross-origin resource sharing     |
| dotenv       | ^16.4.5     | Environment variable management   |
| Nodemon      | ^3.1.4      | Auto-restart dev server           |

## Dev Tools

- Git/GitHub — Version control and collaboration
- VS Code — Code editor
- Postman/cURL — API testing
- MongoDB Atlas — Cloud database hosting
- Chrome DevTools — Debugging and responsive testing

## 7. Database Design

### 7.1 User Collection

**Sample document:**

```
{ "_id": "ObjectId", "name": "string", "email": "string (unique)", "password": "string (bcrypt)",  
  "createdAt": "Date", "updatedAt": "Date" }
```

### 7.2 Task Collection

**Sample document:**

```
{ "_id": "ObjectId", "user": "ObjectId (ref: User)", "title": "string", "description": "string",  
  "deadline": "Date", "priority": "enum", "status": "enum", "createdAt": "Date", "updatedAt":  
  "Date" }
```

### 7.3 Database Connection

The backend connects to MongoDB using Mongoose with two modes:

Production: MongoDB Atlas cloud cluster (connection string stored in .env)

Development: Falls back to mongodb-memory-server if local MongoDB is unavailable (data resets on server restart)

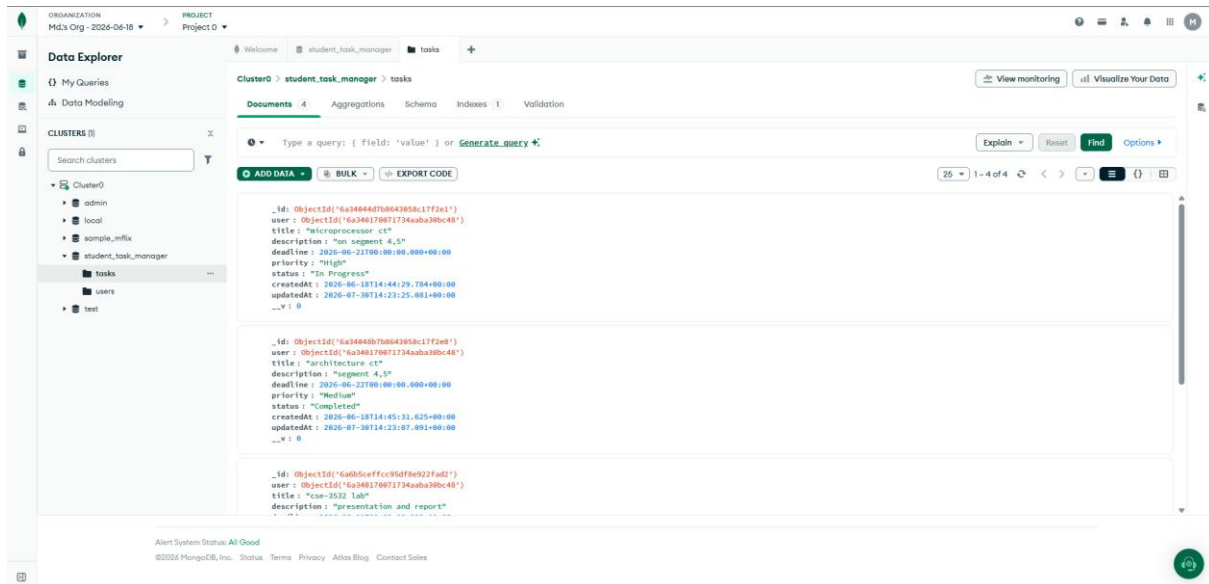


Figure: MongoDB Atlas Cluster — student\_task\_manager database

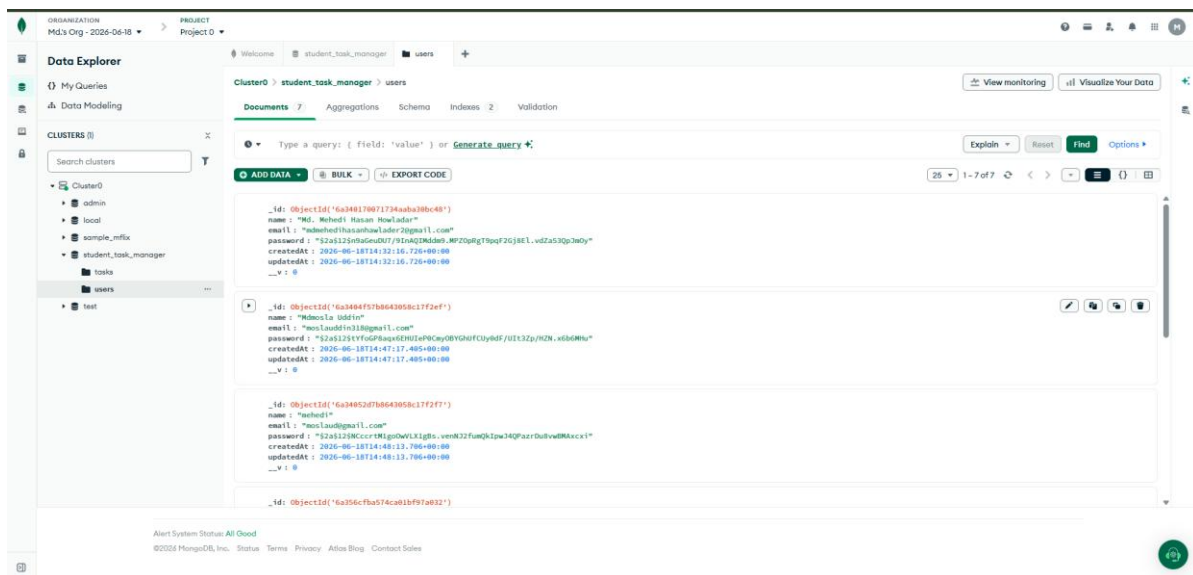
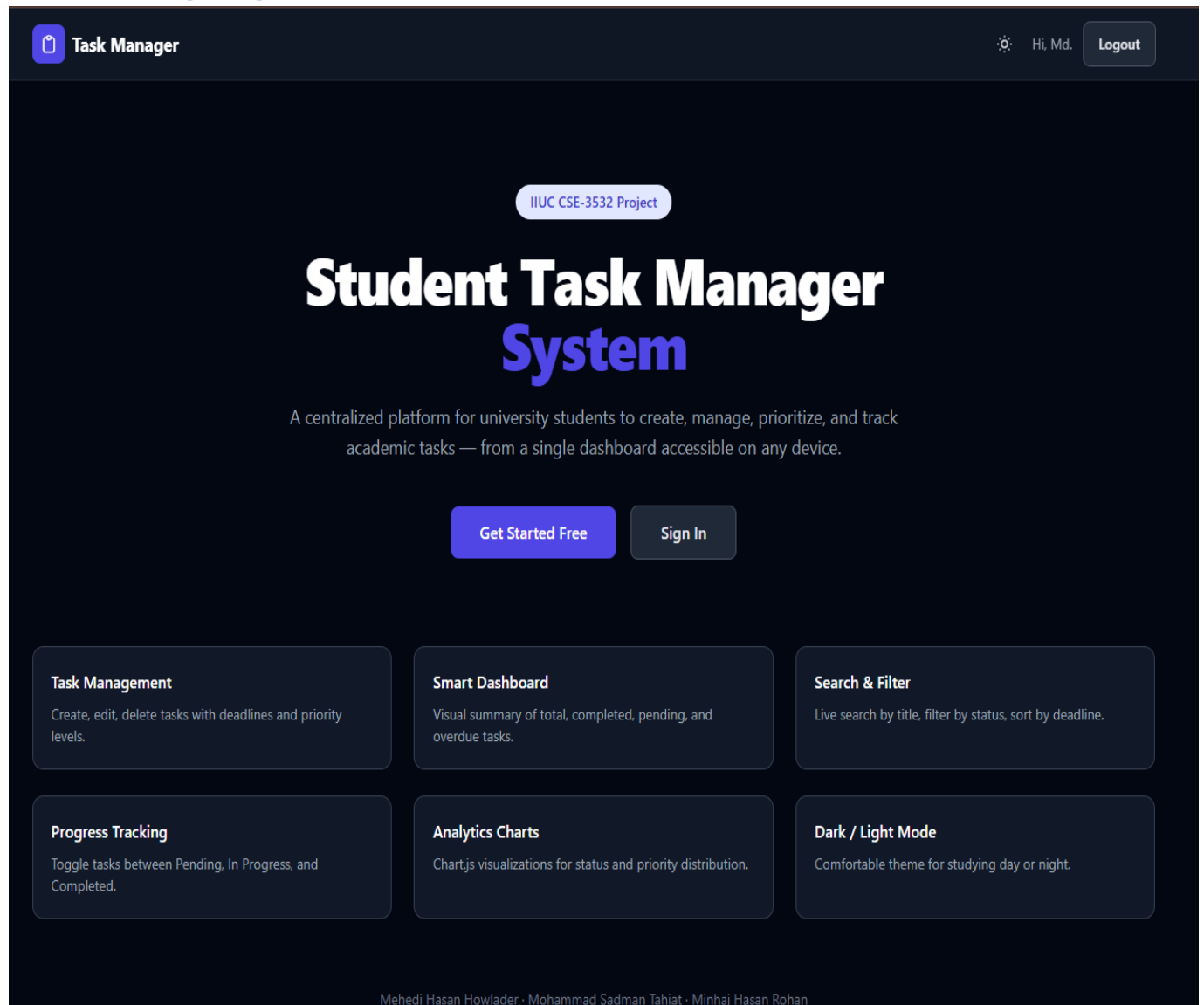


Figure: MongoDB Atlas — users collection

## 8. Screenshots of Project Pages

### 8.1 Landing Page



*Figure: Landing page with hero section and feature cards overview.*

## 8.2 Dashboard (Home Page)

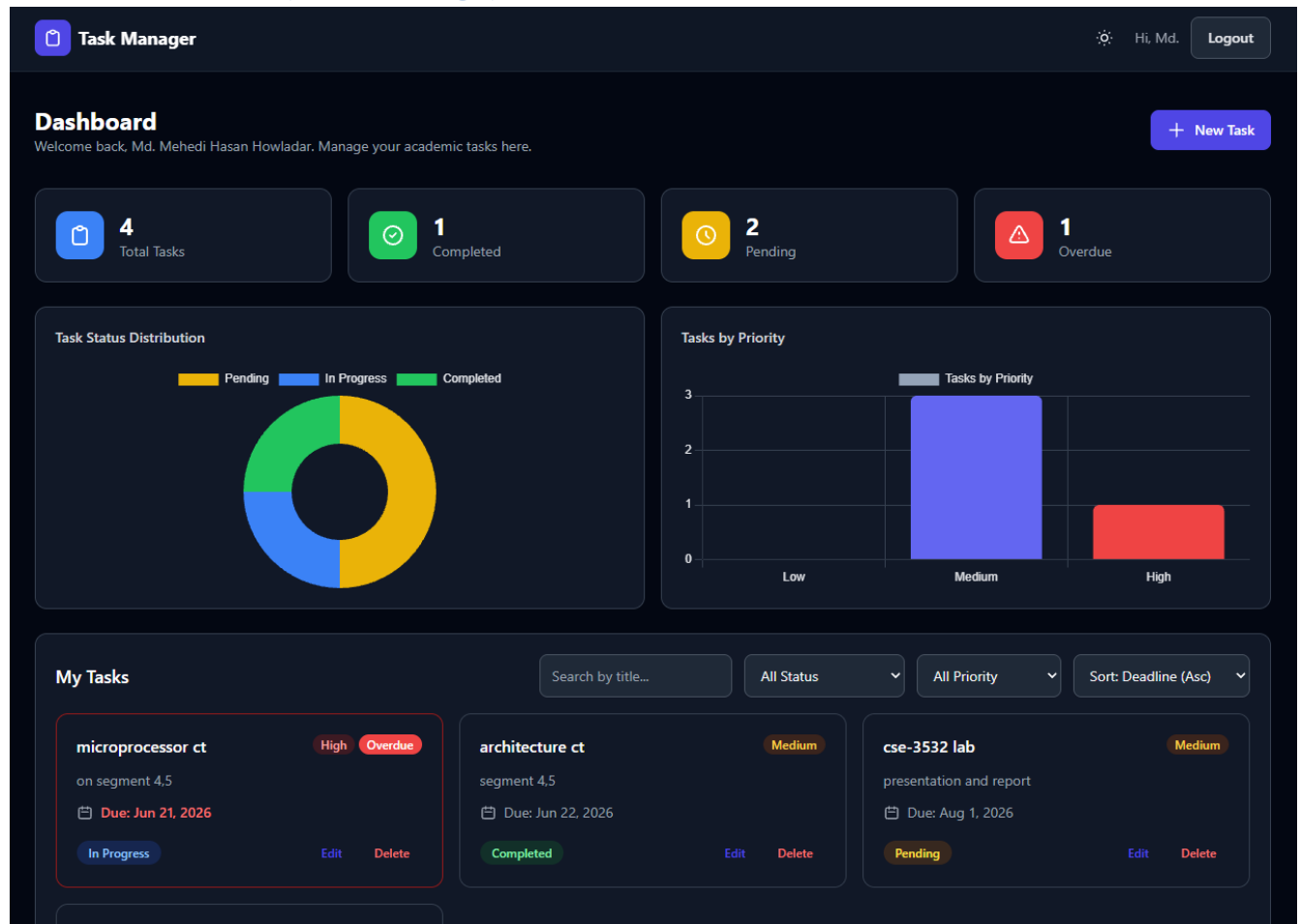


Figure: Dashboard with summary cards, charts, search, filters, and task list.

## 8.3 Dashboard (Light Theme)

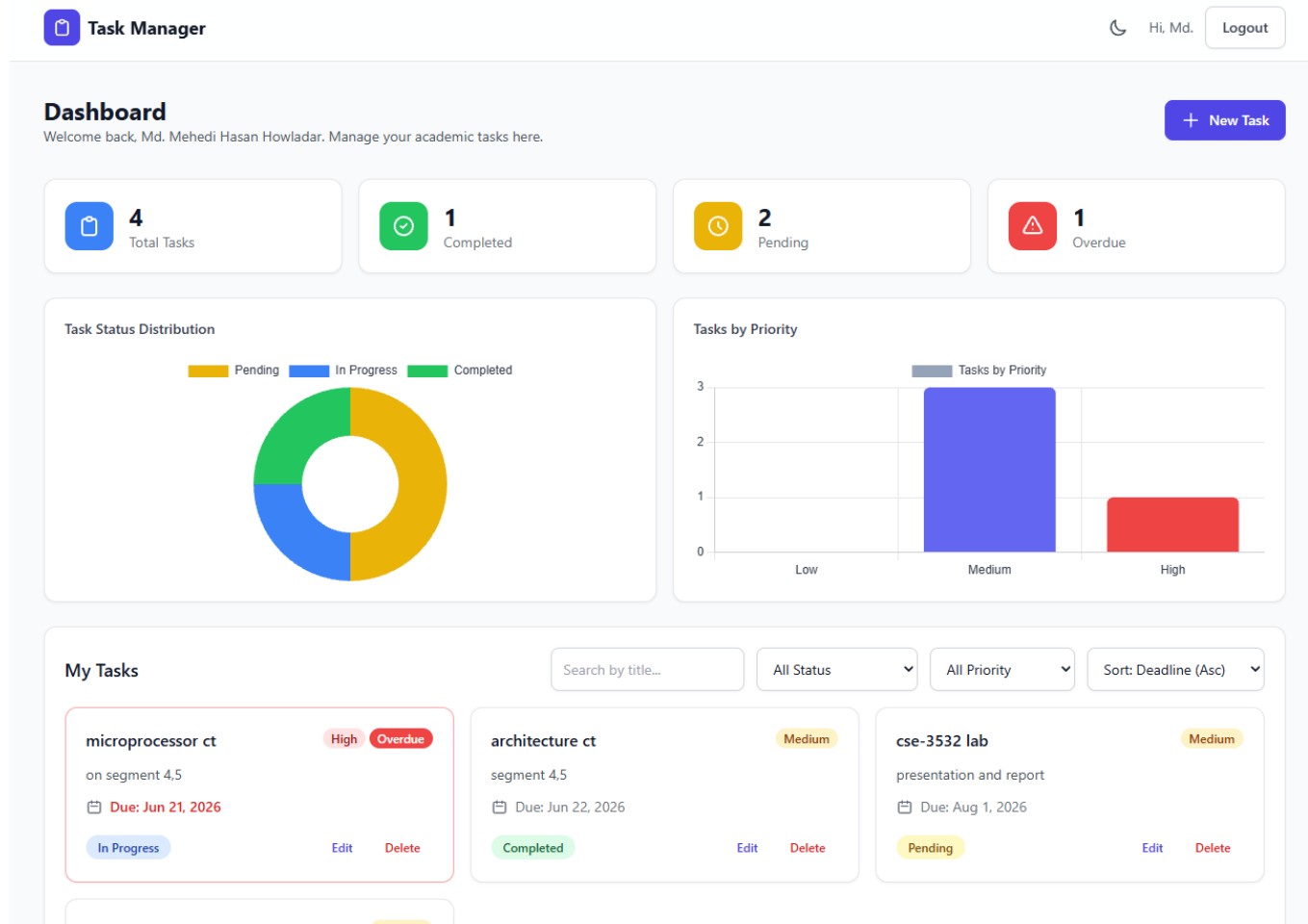
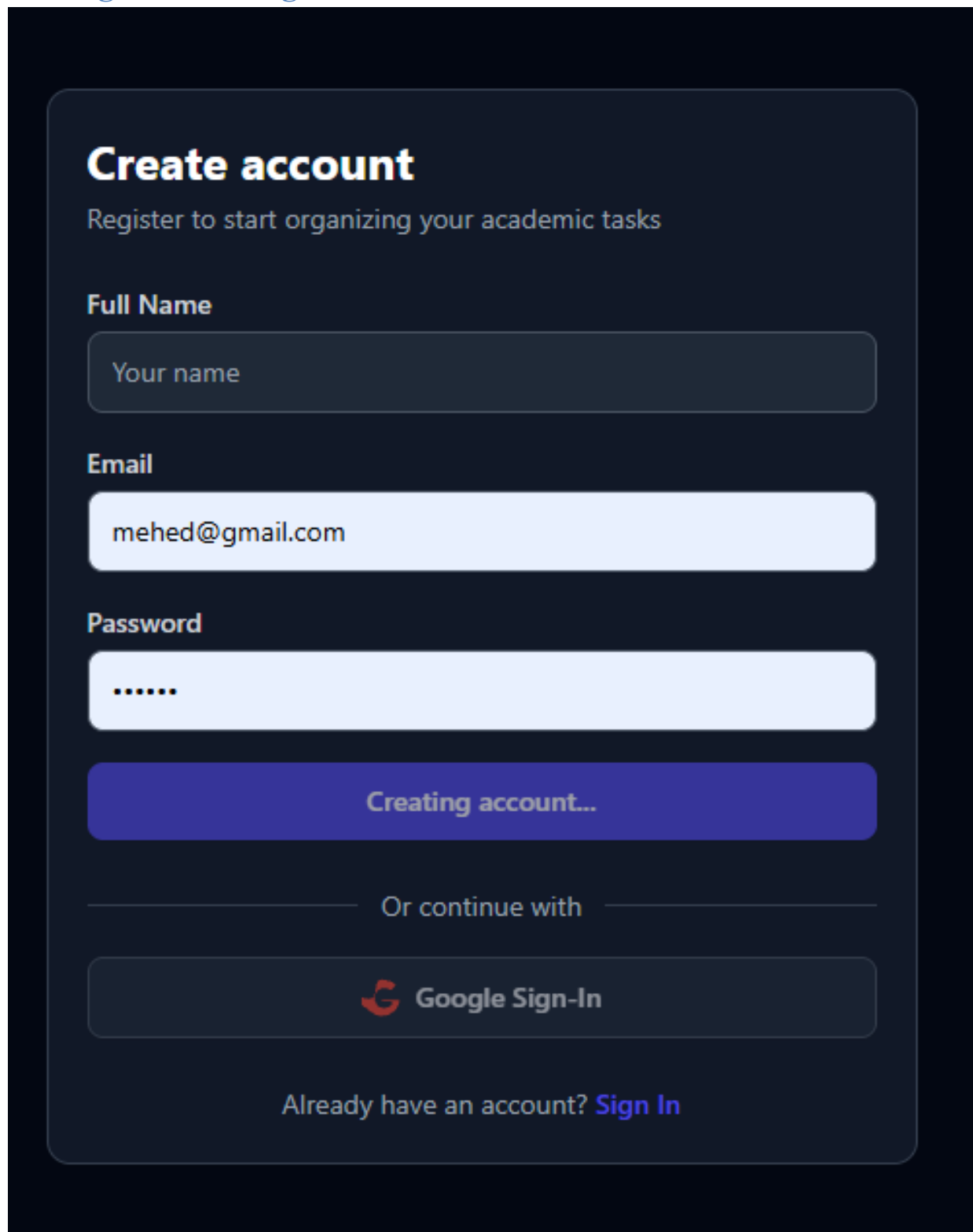


Figure: Dashboard in light theme mode.

## 8.4 Registration Page



The image shows a user registration page with a dark blue background. The main heading is "Create account" in white, bold text. Below it is a subtitle "Register to start organizing your academic tasks" in a smaller, lighter font. The form consists of three input fields: "Full Name" with the placeholder "Your name", "Email" with the value "mehed@gmail.com", and "Password" with masked characters ".....". A blue button labeled "Creating account..." is positioned below the password field. Below this button is a horizontal line with the text "Or continue with" in the center. Underneath is a button for "Google Sign-In" featuring the Google logo. At the bottom, there is a link that says "Already have an account? Sign In".

**Create account**

Register to start organizing your academic tasks

**Full Name**

Your name

**Email**

mehed@gmail.com

**Password**

.....

Creating account...

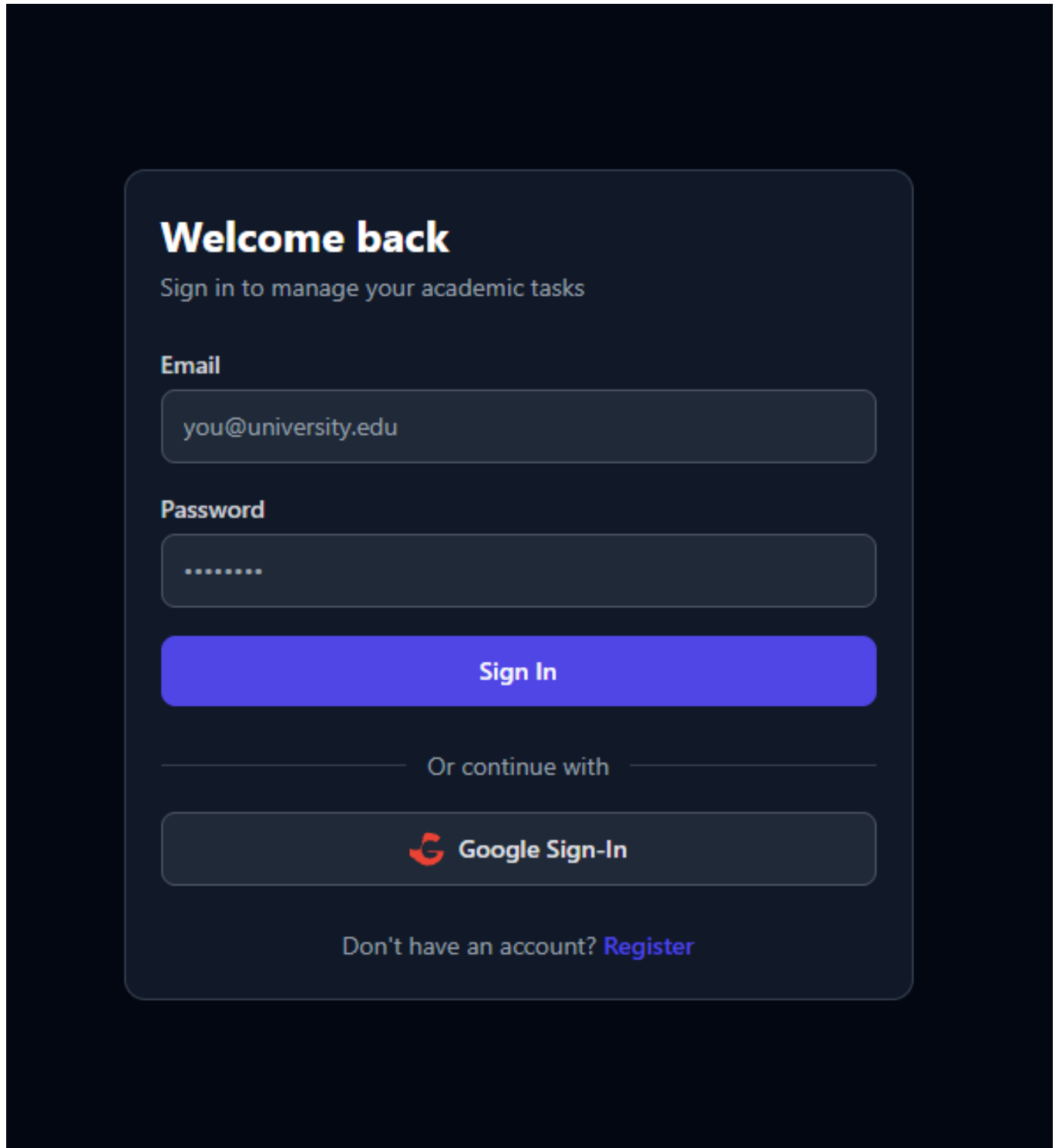
Or continue with

 Google Sign-In

Already have an account? [Sign In](#)

Figure: User registration page with name, email, and password fields.

## 8.5 Login Page

A dark-themed login page with a central card. The card has a title 'Welcome back' in white, followed by the subtitle 'Sign in to manage your academic tasks'. Below this are two input fields: 'Email' with the placeholder 'you@university.edu' and 'Password' with a masked password '.....'. A prominent blue 'Sign In' button is positioned below the password field. A horizontal line with the text 'Or continue with' is centered below the button. Underneath is a button for 'Google Sign-In' featuring the Google logo. At the bottom of the card, the text 'Don't have an account?' is followed by a blue link 'Register'.

**Welcome back**

Sign in to manage your academic tasks

**Email**

you@university.edu

**Password**

.....

**Sign In**

Or continue with

 **Google Sign-In**

Don't have an account? [Register](#)

*Figure: Login page with email/password form and Google Sign-In option.*

## 8.6 Firebase Authentication (Google Sign-In)

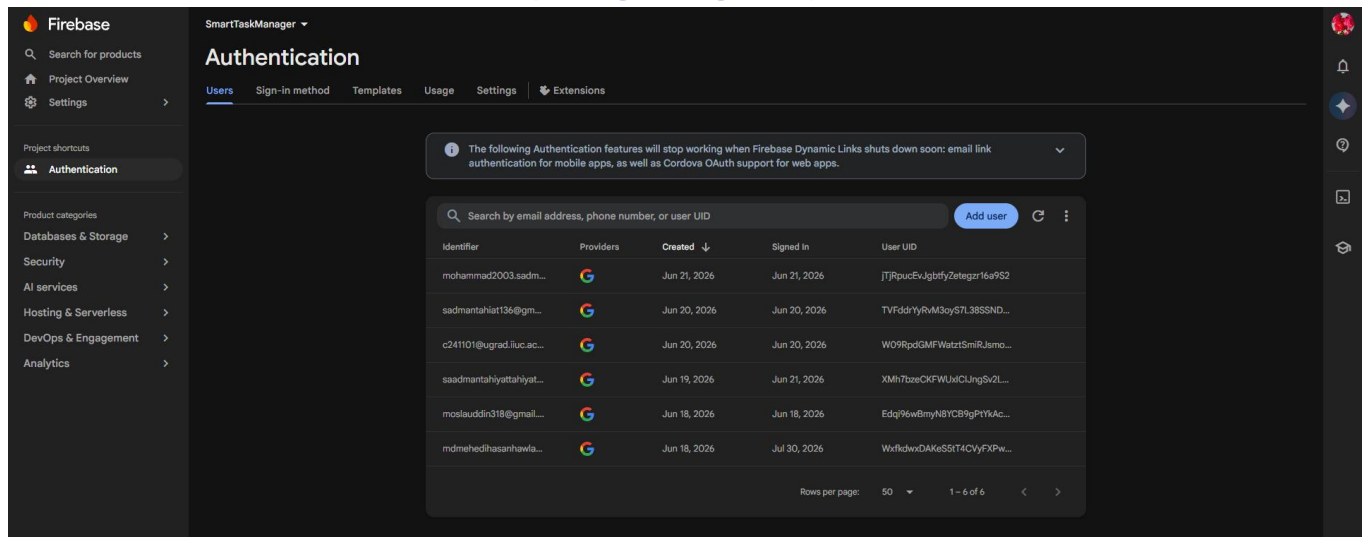


Figure: Google sign-in via Firebase Authentication.

## 8.7 Create Task

The screenshot shows a 'Create New Task' modal form. It has a title field, a description field, a deadline field, and a priority dropdown. The form is set against a dark background with a blurred image of a person.

**Create New Task**

**Title \***

Assignment title

**Description**

Task details...

**Deadline \***

mm/dd/yyyy

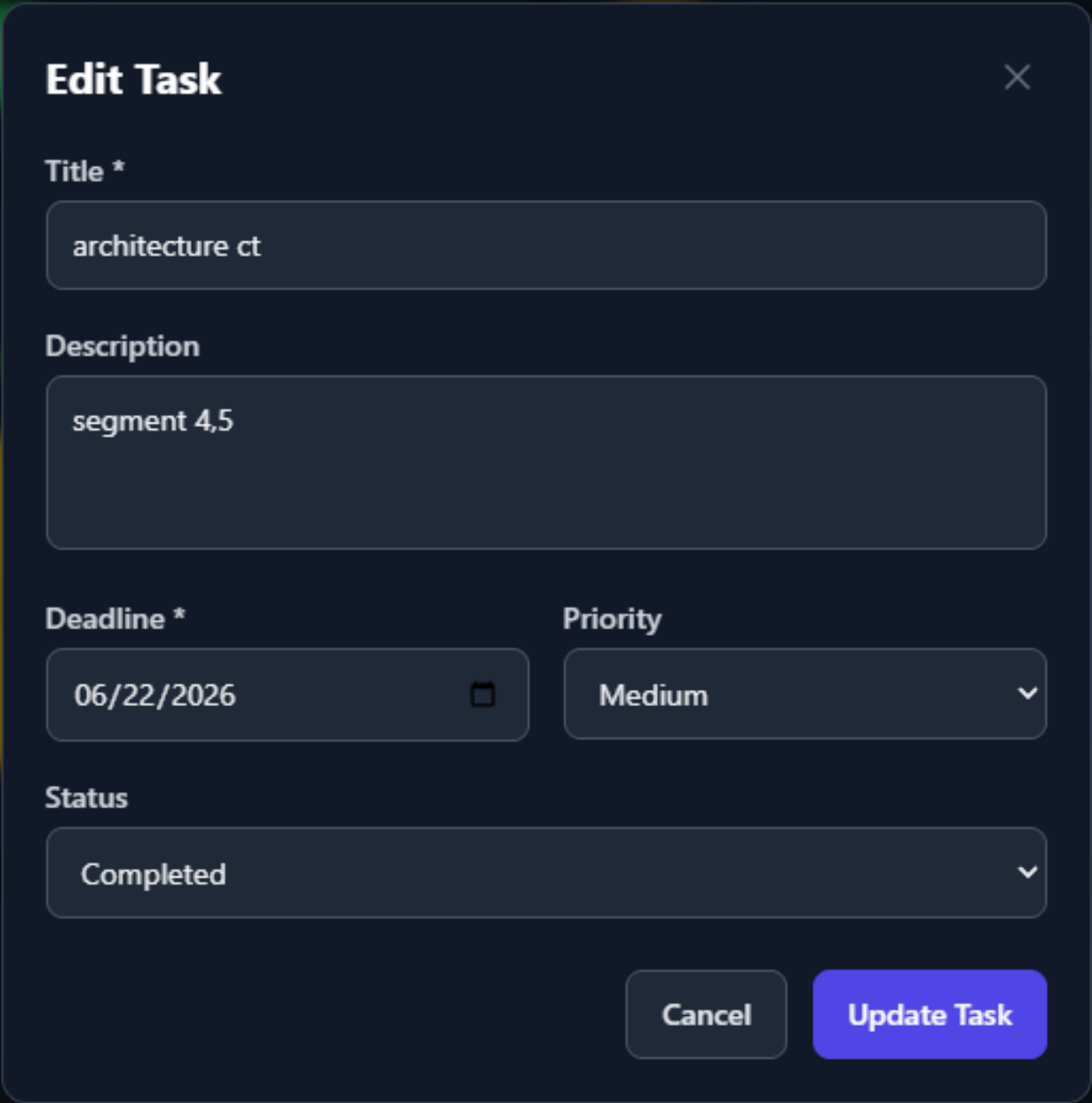
**Priority**

Medium

Cancel Create Task

Figure: Creating a new task via the modal form with title, description, deadline, and priority.

## 8.8 Edit Task



The image shows a dark-themed modal window titled "Edit Task" with a close button (X) in the top right corner. The form contains the following fields:

- Title \***: A text input field containing "architecture ct".
- Description**: A text input field containing "segment 4,5".
- Deadline \***: A date input field containing "06/22/2026" with a calendar icon to its right.
- Priority**: A dropdown menu showing "Medium" with a downward arrow.
- Status**: A dropdown menu showing "Completed" with a downward arrow.

At the bottom right of the modal are two buttons: a "Cancel" button and an "Update Task" button.

*Figure: Editing an existing task.*

## 9. Limitations

- No real-time notifications — the system does not send email or browser push notifications for approaching deadlines or task updates.
- No calendar view — tasks are listed in a grid; there is no calendar-based visualization of task timelines.
- No file attachments — users cannot upload documents, images, or notes within task cards.
- Single user scope — the application is designed for individual use; no collaboration or shared task features for group projects.
- No offline support — the application requires an active internet connection.
- No email verification — user registration does not require email verification, which could allow fake accounts.
- Limited analytics — the dashboard shows basic counts and distribution charts; advanced insights (trends over time) are not available.
- MongoDB Memory Server data loss — when using the in-memory fallback, all data is lost when the server stops.
- No API rate limiting — the API does not implement rate limiting, a potential security concern in production.
- No explicit input sanitization — additional XSS and injection protection beyond Mongoose schema validation is not implemented.

## 10. Future Scope

**Smart Deadline Alerts:** Integrate email notifications (nodemailer) and browser push notifications (Service Workers) for upcoming deadlines.

**Calendar View:** Build an interactive monthly/weekly calendar component showing task density, deadlines, and timelines.

**Collaboration Boards:** Add shared task boards for group projects, enabling team members to delegate and track subtasks.

**AI Priority Assistant:** Integrate a rule-based or ML model to analyze deadlines, descriptions, and history for priority suggestions.

**Document Attachments:** Allow uploads of lecture slides, notes, PDFs, screenshots inside task cards using GridFS or Firebase Storage.

**React Native Mobile App:** Develop a mobile companion with offline-first synchronization using AsyncStorage and background sync.

**Localization / i18n:** Implement multi-language support starting with Bengali translation for IIUC's Bengali-medium students.

**Admin Panel:** Create an instructor dashboard for course-level task completion statistics and oversight.

**API Security Hardening:** Implement `express-rate-limit`, `Helmet.js`, and `Joi`-based input validation for production readiness.

**Email Verification & Password Reset:** Add email verification on registration and a password reset flow using `nodemailer` or `SendGrid`.

## 11. Conclusion

The Student Task Manager System is a fully functional, full-stack MERN web application that successfully addresses the real challenge of academic task management for university students. By integrating secure authentication (email/password and Google OAuth), complete CRUD task management, interactive analytics with `Chart.js`, a responsive dark/light theme UI, and smart search/filter/sort capabilities, the system provides a cohesive and modern task management experience.

The project demonstrates the practical application of modern web development technologies: React for the frontend, Node.js and Express for the backend, MongoDB with Mongoose for data persistence, Tailwind CSS for responsive styling, Firebase for Google authentication, and `Chart.js` for data visualization. It follows best practices in code organization, modular architecture, and security (`bcrypt` password hashing, JWT authentication, protected routes).

While there are limitations — such as the lack of real-time notifications, calendar view, and collaboration features — the current scope delivers a solid foundation that can be extended with the future enhancements outlined in this report. The project successfully fulfills the requirements for the CSE-3532 course and serves as a learning reference for full-stack web development using the MERN stack.